

Hi,

Thank you for taking the time to attend the first interview session with us. We appreciate the effort you put into the discussion and getting to know our team and context.

We wish you the very best of luck with the next steps and this task. We're looking forward to reviewing your work and continuing the conversation.

Context

You are working in a travel-tech company. The data platform is a lakehouse architecture where analytical queries are served from analytical tables built from operational data.

You are given raw booking-related datasets that simulate data coming from a CDC pipeline (e.g., Debezium → Kafka → Lakehouse). Your job is to design and implement a **Bronze** → **Silver** → **Gold** data flow and expose reliable analytics on top of it.

You may use **SQL only** (Trino/Spark SQL style) or **dbt** if you prefer. You do **not** need to deploy anything; local execution and plain files are sufficient.

Data

You are given three raw tables (CSV or JSON format).

1. **bookings_raw**

Represents the mutable state of bookings (multiple updates per booking).

Columns:

- **booking_id** (string)
- **user_id** (string)
- **hotel_id** (string)
- **status** (created, confirmed, cancelled)
- **price** (decimal)
- **created_at** (timestamp)
- **updated_at** (timestamp)

Notes:

- A booking may appear multiple times with different **updated_at**.
- Status can change over time (e.g., confirmed → cancelled).

2. booking_events_raw

Represents event-level changes (append-only).

Columns:

- `booking_id` (string)
- `event_type` (created, confirmed, cancelled)
- `event_ts` (timestamp)

Notes:

- Events can arrive late.
 - Multiple events may exist for the same booking.
-

3. hotels_raw

Reference data.

Columns:

- `hotel_id` (string)
 - `city` (string)
 - `star_rating` (int)
-

Your Tasks

Part 1 – Silver Layer (State Modeling)

Build a **Silver** table that represents the **latest correct state per booking**.

Requirements:

- One row per `booking_id`
- Must correctly handle:
 - multiple updates
 - late-arriving events
 - cancellations

- Clearly define:
 - what timestamp determines “latest”how conflicts between `bookings_raw` and `booking_events_raw` are resolved

Deliverables:

- SQL/dbt model for the Silver table
 - Short explanation of your logic and assumptions
-

Part 2 – Gold Layer (Analytics Mart)

Build a **Gold** analytics table for **daily booking KPIs**.

Grain:

- One row per **day × city**

Metrics:

- total bookings count
- confirmed bookings count
- cancelled bookings count
- cancellation rate
- total revenue (only confirmed bookings)
- average booking price

Requirements:

- Metrics must be consistent with booking state logic
- Clearly define which date is used (`created_at`, `event_ts`, or something else)
- Avoid double counting

Deliverables:

- SQL/dbt model for the Gold table
 - Explanation of metric definitions
-

Constraints & Expectations

- Assume a distributed query engine (e.g., Trino or Spark)
 - Assume tables are stored in a lakehouse format (Iceberg/Delta)
 - Focus on **correctness, clarity, and tradeoffs**
 - Over-engineering is discouraged; explain decisions instead
-

What to Prepare for the Presentation

During the onsite session, you should be ready to:

- Walk through your data model
 - Explain edge cases (late data, duplicates, cancellations)
 - Discuss performance and cost implications
 - Answer “what if” scenarios (schema changes, reprocessing, backfills)
-

Submission

Please provide:

- SQL files or dbt project
 - A short README explaining:
 - assumptions
 - design decisions
 - known limitations
-

This task is intentionally open-ended. There is no single correct solution. We are evaluating **how you think**, not just what you write.